

University of Toronto

CSC490

Final Report

Name	Student Number
Yusuf Moola	1007658777
Mikhail Skazhenyuk	1009376337
Rudraksh Monga	1008018342
Codi Lee	1007698608

1 Abstract

The Formula 1 sponsorship market generates over \$2 billion annually [1], yet sponsors lack precise metrics to calculate their return on investment. Brands pay premium rates based on estimated viewership and subjective placement assessments, with no quantitative measurement of actual screen time. We address this gap by developing an end-to-end computer vision system that automatically detects and quantifies brand logo exposure in F1 race footage with frame-level precision.

Our solution centers on a fine-tuned YOLOv8 object detection model trained on a curated dataset of 3,049 annotated F1 frames containing 86 distinct sponsor brands. We implement a serverless inference architecture on Modal Labs that processes videos asynchronously, extracting frames at configurable rates, performing batch inference on NVIDIA T4 GPUs, and generating comprehensive exposure metrics including visibility duration, screen area, and per-brand analytics. The system uses Amazon S3 as a versioned data lake for reproducible training pipelines and stores all artifacts—datasets, model weights, and inference results—with built-in disaster recovery through cross-region replication.

Through systematic optimization of frame extraction, we achieved a significant reduction in inference latency by compiling model weights to NVIDIA TensorRT engines with FP16 quantization and dynamic shape support. Furthermore, we eliminated disk I/O bottlenecks by engineering a streaming in-memory inference pipeline. This architecture bypasses intermediate disk storage by feeding decoded video frames directly from RAM to the GPU in optimized batches. This approach, combined with batch size clipping to prevent CUDA Out-of-Memory (OOM) errors on Tesla T4 GPUs, scales effectively from single-user development to production workloads. Our architectural design demonstrates practical trade-offs between model accuracy, inference latency, and operational cost across 10×, 100×, and 1000× user growth scenarios.

This work establishes a production-ready framework for automated sports sponsorship analytics, replacing manual measurement with precise, data-driven insights that enable evidence-based negotiation and portfolio optimization for brands investing in motorsports marketing.

2 Introduction

2.1 Problem Statement

Sports sponsors lack precise metrics to verify their return on investment. In Formula 1, brands pay premium rates for logo placement based on estimated viewership and subjective assessments, with no quantitative measurement of actual screen time. This measurement problem is particularly acute in F1 due to dynamic camera angles, motion blur, occlusion, and varying scales as logos appear on high-speed vehicles. Manual frame-by-frame analysis is prohibitively expensive, while existing commercial platforms, such as Relometrics [25], Blinkfire [26], and Sponsolytix [27] provide proprietary solutions not specialized for motorsports footage.

We address this gap by developing a domain-specific computer vision system that automatically detects and measures brand logo exposure in F1 race videos with frame-level precision. Our system processes race footage, identifies 86 distinct sponsor brands, and generates quantitative exposure metrics including visibility duration, screen area percentage, and temporal distribution.

The technical challenges span multiple dimensions. First, logo detection requires robustness to extreme motion blur, perspective distortion, and partial occlusion. The modeling aspect is an object detection and image segmentation problem. Second, the system must process hours of high-resolution video with acceptable latency and cost through optimized frame extraction, batched GPU inference, and cloud storage operations. Third, the infrastructure must scale gracefully while maintaining reliability under adversarial conditions.

Our solution uses a fine-tuned YOLOv8 model trained on 3,049 annotated F1 frames. F1 introduces domain-specific complications: logos vary dramatically in size, appear under challenging lighting conditions, and must be distinguished from similar graphical elements. We augment training data with photometric and geometric transformations to improve robustness. We prioritize this closed-set detection approach because F1 sponsors are well-defined and domain-specific models offer superior cost-performance tradeoffs - essential when inference costs scale linearly with frame count.

Beyond modeling, our system architecture addresses practical deployment concerns. We implement a serverless inference service on Modal Labs with asynchronous job processing and GPU autoscaling. The system uses Amazon S3 as a versioned data lake with cross-region replication. Through systematic load testing [28], we identify and mitigate critical vulnerabilities including GPU memory exhaustion and cascading failures.

Our contributions include: (1) a production-ready F1 logo detection pipeline with comprehensive data augmentation, (2) systematic optimization of video processing throughput through parallel operations and GPU memory management, (3) infrastructure design patterns with cost-performance analysis at 10 $\times$, 100 $\times$, and 1000 $\times$ growth scenarios, (4) adversarial load testing methodology with practical mitigations, and (5) reproducible training and deployment pipelines with automated CI/CD and disaster recovery.

The remainder of this report reviews related work (Section 2), describes our methodology (Section 3), presents experimental results (Section 4), discusses limitations and future work (Section 5), and concludes with project reflections (Section 6).

3 Related Work

3.1 Academic

The modelling aspect of our problem statement is a combination of object detection and image segmentation. Traditional object detection tasks that detect object presence and location at the single image level is considered a fairly solved problem [2]. Notably, the YOLO series models from Ultralytics have established a strong baseline for boxed object detection with a fast and light architecture [3, 13]. More recently, there have been several transformer-based image object-detection models [6, 7]. The primary caveat for the traditional object detection task is that it is a supervised model which requires a task-specific labeled dataset and must be retrained on new labels [2]. The current leading models aim to address this by integrating object detection backbones with expanded or "open" label sets, called the Open World Object Detection task [2]. They often integrate linguistic datasets and natural language reasoning capabilities to approximate these "open-set" labels [4, 5].

The segmentation problem aims to segment at a more precise level than boxed detections. Most notably within this space are the SAM models [8, 9], which are frequently used as a backbone given its zero-shot and promptable capabilities. A prompt for SAM models are not natural language, but are boxes or point indicators. Our task does not demand the additional precision offered by true segmentation models since most labels follow a rectangular shape, so we stayed with the simple YOLO architecture backbone.

There is also a body of work that pipelines object detection predictions into image segmentation models to create better context targeted image segmentations [10, 11, 12]. VLMs are commonly employed for these flexible context detection and segmentation models, which rely on the semantic reasoning capabilities of natural language reasoning systems to determine semantically relevant segments of the image to box [11, 12]. This allows for zero-shot generalization for a broader range of concepts. For example, our project task could be framed as a generic logo detection engine rather than trained to detect specific logos from specific brands. Since the concept of logos are widely diverse, we saw some potential benefit in integrating these VLM-based methods into our task which is mentioned in the Methodology section.

Finally, our modelling problem could also be taken in a video-processing perspective that considers frame sequencing to generate aggregate insights. Video processors typically involve some context integration method (e.g. transformer for SAM2 [9], frame sampling, or 3D convolutions) [15]. Other video-processing methods focus on improving the speeds of image-level processes to speed the overall video [14]. As it pertains to our problem statement, video-processing methods that integrate frame-adjacent temporal context might result in more accurate models which can track objects into blurred and angled frames.

3.2 Industry

Computer vision methods have long been used as tools for businesses to process inventory, categorize content, and more (ex. [16]). For advertising and marketing, there are companies that specialize in creating image segmentation and object detection tools to analyze and learn to improve advertising strategies (ex. [17]). Computer vision for video processing has also been increasing in prominence in sports events. In the specific space of f1 racing, the 2023 Abu Dhabi Grand Prix used computer vision methods to automatically label and detect track violations to ease the load off human experts [18].

Specifically within advertising, segmentation and object detection methods have been used to create CV branding exposure analysis tools. For example, Relo Metrics and Blinkfire boast a unified analysis platform for sports-based advertising: detecting gameplay footage across different sports and social media engagement to create unified insights. Sponsorlytix provides a suite of hierarchical sports labeling detector models. The challenge of generalized sports segmentation is the

broadness of the different possible sports resulting in a broad range of contexts for the vision model to learn. We focused this capstone project on the problem of F1 racing brand detection to maximize the ML model quality specifically for F1 racing. Domain-specific models are cheaper and more effective for the domain-specific task than domain-general models 20 (ex. 15). Model maintenance and inference costs are particularly significant for video media forms which can magnitudes more expensive in the simplest case of batching single image inference on many frames, but can exceed this if extracting more meaning from the sequencing of the frames themselves 19. This trade-off between generality and cost is a common consideration in industry usage of LLMs, where many methods are developed to fine-tune smaller units of models for a business-specific use case (e.g. LoRA 22 21). The prominence of ensemble-based methods in the industry underline the effectiveness of model task or data specialization for performance (e.g. Mixture of Experts 23, boosted trees). Adjacently, in order to decrease latencies of real-time ML processes, there has been a growing movement towards combining edge computing with AI use-cases; Edge AI 24. This is particularly key for streamed video processes which requires a lot of engineering to be able to serve predictions at commercially acceptable latencies. However, Edge AI only applies if true model prediction streaming is strictly necessary. For our use case, brand exposure can be resolved within a day without much impact to the business, and there is little sense of "bringing the model closer to the source data" since the insights are from derived video content that is already being streamed across the world.

4 Methodology

4.1 Solution

We implemented an end-to-end computer vision pipeline for automated F1 logo detection and exposure measurement. This addresses the previously addressed critical gap in sports sponsorship analytics by providing precise, quantitative metrics that replace subjective manual assessments.

4.1.1 Core

There are three primary components: (1) Fine-tuned YOLOv8m object detection model trained on F1-specific logos, (2) server-less inference service on Modal Labs with GPU auto scaling, and (3) versioned data lake on Amazon S3 for reproducible training and artifact management.

4.1.2 Model Training Pipeline

Initially we started with the Roboflow F1 Logo Dataset containing 3,049 annotated frames with 86 distinct sponsor brands. The raw dataset had a significant imbalance, with some brand appearing less than 10 frames and others appearing over 200. We addressed this by:

1. **Stratified Splitting:** Iterative multi-label stratification to ensure fair representation in splits.
2. **Targeted Augmentation** applied to under-represented classes, including photometric adjustments, geometric transformations, and realistic degradations.
3. **Data Versioning** with complete reproducibility stored in S3.

The YOLOv8m model was selected for the excellent speed-accuracy trade-off compared to transformer based alternatives, especially considering the costs of vision models. The model was fine-tuned on the augmented dataset with Modal, with all artifacts automatically exported to S3 for versioning and disaster recovery.

4.1.3 Inference Service

The inference pipeline operates asynchronously to handle the high latency inherent in video processing.

1. **Job Submission:** Users submit a video.
2. **Queue:** Jobs enter a FIFO queue, status tracking being available.
3. **TensorRT Optimization:** The YOLOv8m model is compiled into a TensorRT engine using FP16 half-precision and dynamic batch sizes. This reduces memory footprint and maximizes throughput on the T4 GPU tensor cores.
4. **In-Memory Streaming:** Instead of writing raw frames to disk (a high-latency operation), the system utilizes a generator-based pipeline. Video frames are decoded into RAM buffers and passed directly to the GPU.
5. **Batched Inference & Annotation:** Frames are processed in batches (e.g., size 30-80) managed by strict memory constraints to ensure stability on 16GB VRAM. Post-processing and annotation occur immediately in memory using optimized C++ plotting routines before results are aggregated.

6. **Post-processing:** Detections are filtered by confidence, bounding boxes are drawn, and frames are annotated.
7. **Results:** Summary statistic are given.
8. **Storage and Retrieval:** Annotated frames and JSON results uploaded to S3 and downloadable via API.

4.2 Technical

4.2.1 Tooling and Infrastructure

S3: Data Lake

Our S3 was chosen to be used as our data lake. It stores all artifacts of our project:

- Model datasets (train, validation, test) and augmented variants
- Model weights and training results
- Configurations for model training runs and datasets
- EDA results
- Inference videos and results on request

While the final user workflow only involves uploading videos for analysis, the training workflow is far heavier: we manage thousands of labeled images, augmented variants, synthetic samples, and multiple model checkpoints. Storing these directly in Modal would be brittle, so instead we use S3 as our data lake for a durable source of truth. S3 holds raw exports, processed splits, validation reports, and model artifacts in a versioned layout. This ensures reproducibility across experiments and team members. It also allows downstream inference services on AWS to directly consume the exported models and configuration files from the same repository, avoiding manual file transfers between training and serving environments.

In addition, we also chose S3 for its great support on data redundancies to ensure that our service can remain live for long periods of time. Our data lake is versioned and replicates its data to a separate S3 region in case one region collapses. The region system of S3 storage also itself has redundant copies of the data within the region.

Modal: IaC and Serverless Applications

We considered the expected traffic patterns to select our compute tooling. On average, there are F1 races evenly spread throughout the year at intervals of every 3 weeks. Requests to our model will surge leading up to and following from these races. The brands that participate in sponsorships are in the order of hundreds, but not in the thousands CITE, so request rates during the surge periods will also likely remain in the low thousands per day. Additionally, the hundreds of companies participating are from around the globe, leading to a fairly variable range of active periods. A successful model service should have the properties that can support regular periods of request surges without collapsing, and be available at all times in the day at a moment's notice.

We found that Modal's serverless application structure suited our needs for supporting scattered select customers from around the world who buy access to the application server. The serverless application design ensures that resources are not wasted during periods of low usage, and can autoscale with similar ease when necessary. While serverless applications do face some additional coldstart latencies after not receiving any requests for a certain time, since video processing is itself a fairly high latency ML problem, the coldstart times should not have a huge impact to customer experience.

In addition, Modal serves to simplify our infrastructure stack. Modal provides a high level of abstraction for resource allocation and dependency configuration and adds simple integrations with S3.

4.2.2 Architectural Design

The architectural designs are detailed in Figures 1 and 2. Figure 1 details the pipelines for data ingestion and preprocessing for the model training component. We ingest our primary dataset from Roboflow, a Computer Vision dataset and model community website. The raw and derived datasets pass through ingestions into the S3 buckets. These pipelines can be retriggered afterwards for existing datasets. Figure 2 details the pipeline structure of the model server. The dev bucket and server are an internal optimization to ensure that the server always contains robustly tested and functional versions of the code which is mediated by Github Actions.

Figure 3 demonstrates the expected endpoint request structure for users of our model service. Given the high latencies of video processing models, to avoid complications due to network instabilities for the uncertain length of processing, we decided to design our model server as asynchronous: users will submit jobs and later retrieve the results. To break down the process further:

- A user first pass in a public video link into the service which will produce a unique job id.

- The user can then track the progress of the job as it first waits in the FIFO queue to be processed and its completion of model prediction through the check status endpoint.
- Once the job completes, the user is able to download the results with the download results endpoint.

The schemas for the endpoints are detailed in Figure 3.

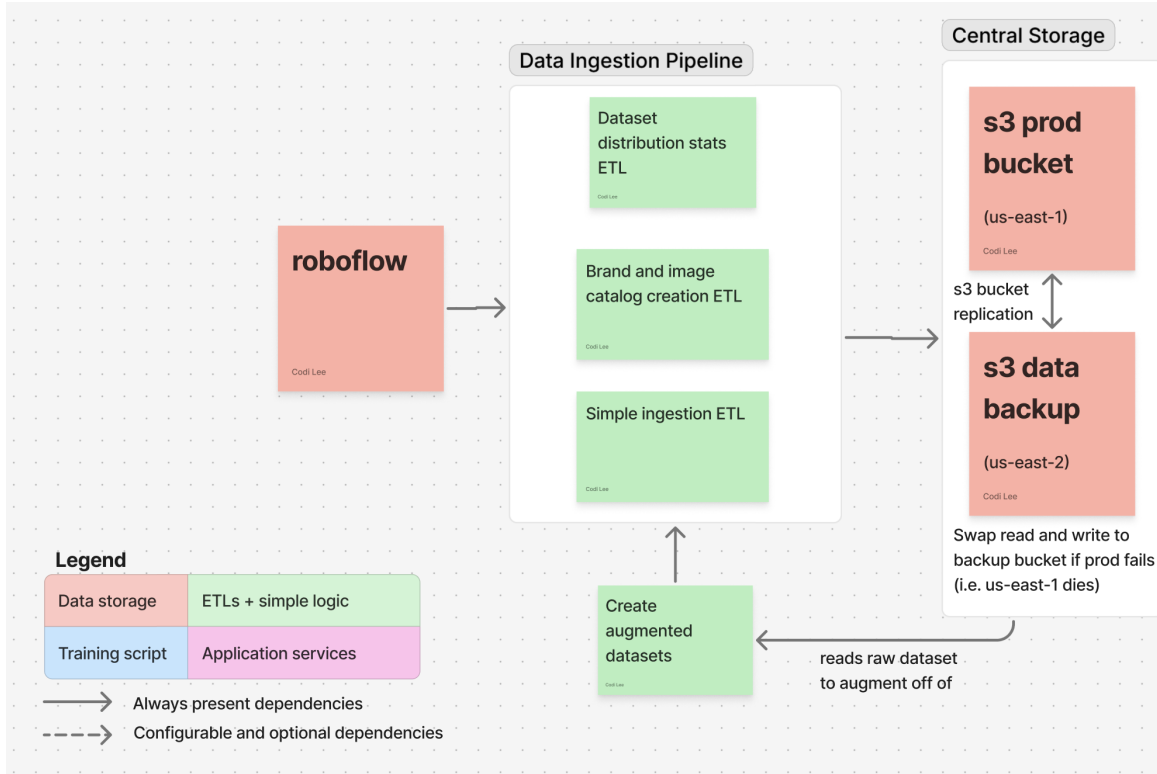


Figure 1: The data processing and ingestion pipeline half of the infrastructure. Red includes data storages which the green ETLs and simple logic modify and save back into the storage buckets. The ingestion ETLs currently run locally, but it is better to transfer them into Modal jobs to handle larger dataset ingestions. Notably, the prod S3 bucket has a replication backup with ETL logic that swaps reading destination in case of prod S3 bucket downtime.

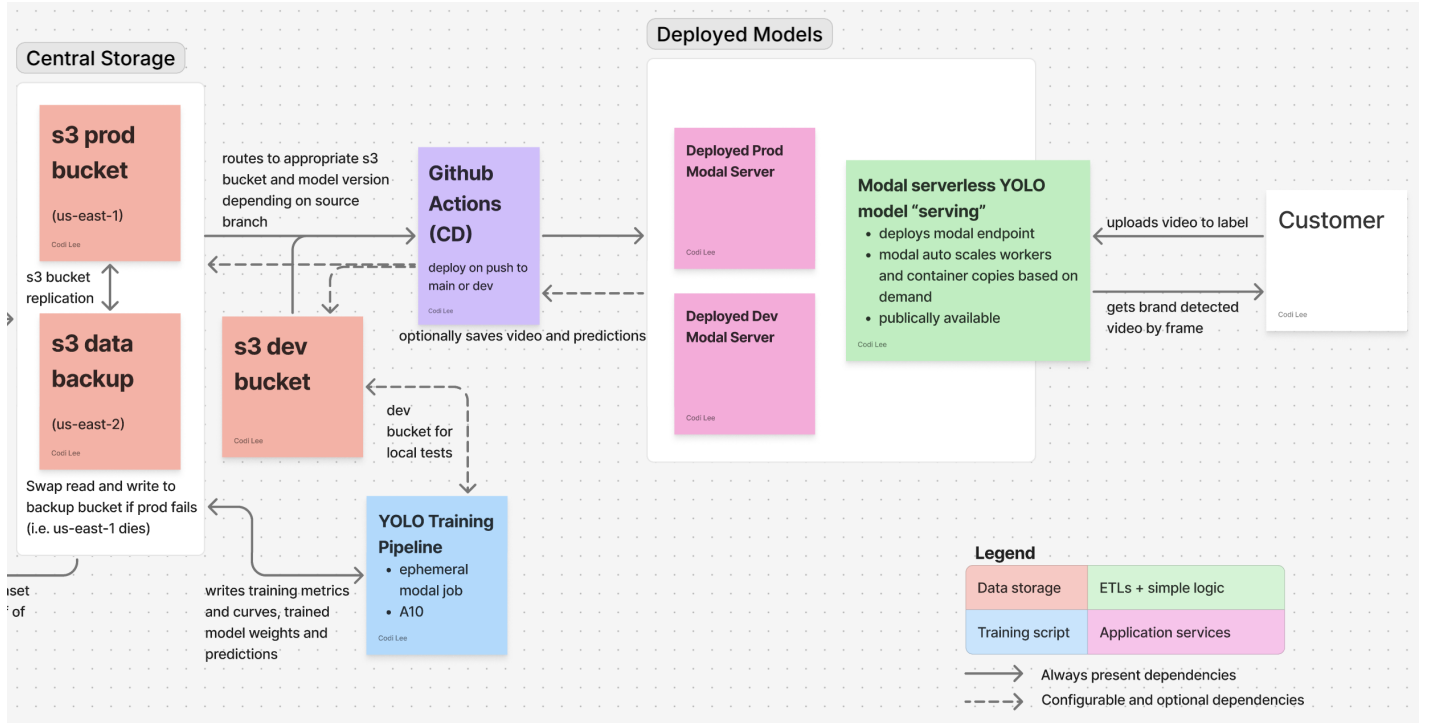


Figure 2: The model training and deploy half of the infrastructure and logic. The training pipeline reads and writes to appropriate S3 buckets to train the YOLO model. This is an ephemeral Modal job. There is a Modal app for deploying the trained model. This diagram treats the app’s script as an IaC configuration since this base code is actually deployed with two copies: dev and prod. This allows the server to be tested freely before any changes might disrupt prod traffic. The Github Actions mediates deployments to each dev and prod application servers automatically as triggered by pushes on github to the dev or main branches. The customer interfaces with the endpoints on the servers; passing in videos to the endpoints and retrieving frame-by-frame labeled responses.

4.2.3 Pipeline Optimizations

We migrated from standard PyTorch inference to NVIDIA TensorRT to leverage hardware-specific optimizations. During this process, we encountered GPU memory limitations (OOM) on the Tesla T4 when processing 640px inputs with large batch sizes. We resolved this by:

- Enabling *dynamic=True* during export, allowing the engine to handle variable batch sizes without reserving maximum VRAM for worst-case scenarios.
- Implementing strict batch-size caps (limiting batches to 40 on T4) to fit within the 16GB VRAM budget while maintaining high CUDA core saturation.
- Utilizing FP16 quantization to effectively halve the memory footprint of model weights and activations.

Removing the I/O Bottleneck

Early profiling revealed that despite TensorRT acceleration, total processing time remained high. The system was disk-bound, spending significant time writing thousands of extracted JPEGs to storage before reading them back for inference. We refactored the pipeline to perform In-Memory Streaming: frames are now decoded, buffered in RAM, and sent to the GPU in a single pass. This removed the read/write overhead entirely, shifting the bottleneck back to the GPU compute, where it belongs.

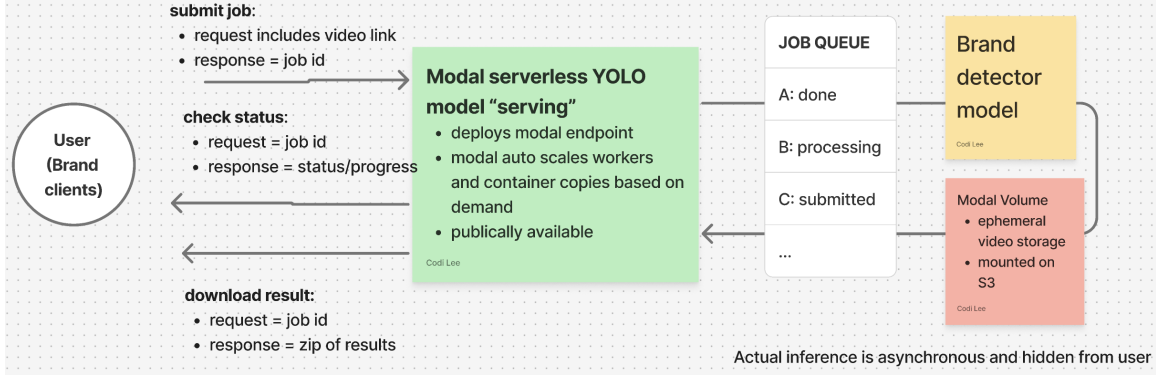


Figure 3: User request structure to the server. There are three primary endpoints for the user: one to submit the job, the other to check the status of the job, and once it is marked completed, an endpoint to download the result. Jobs are buffered and processed in a job queue where upon inference is run. The actual inference is asynchronously run.

5 Results

5.0.1 Modelling

Training Infrastructure and Pipeline

We architected a cloud-native training pipeline utilizing **Modal Labs** to orchestrate high-performance compute resources. The training job was distributed across **three NVIDIA A100 (40GB)** GPUs utilizing Distributed Data Parallel (DDP) strategies to maximize throughput.

To resolve the I/O latency bottlenecks inherent in reading thousands of small image files directly from cloud storage, we implemented an ephemeral data staging strategy. Upon container initialization, the pipeline stages the versioned dataset from our **Amazon S3 data lake** (mounted via FUSE) into the container’s local NVMe storage. This `stage_dataset_from_s3` optimization eliminated network overhead during the training loop, ensuring GPU saturation remained the primary constraint rather than disk I/O.

Experimental Configuration

We selected the **YOLOv8m (Medium)** architecture as our champion candidate, offering an optimal balance between parameter count (25.9M) and inference speed for our resource-constrained T4 inference targets. The model was fine-tuned on the “augmented” dataset version with the following hyperparameters:

- **Epochs:** 120 (with a 3-epoch warmup phase)
- **Image Resolution:** 640×640
- **Batch Size:** 24 (distributed across 3 GPUs)
- **Optimizer:** SGD (stochastic gradient descent) with momentum
- **Freezing:** None (Full fine-tuning of backbone and head)

Quantitative Performance

The training run achieved state-of-the-art convergence for this specific domain. Over 120 epochs, the model demonstrated robust learning characteristics with no signs of significant overfitting.

- **Global Accuracy:** The model achieved a **mean Average Precision (mAP@50)** of **0.926**, indicating that for 92.6% of detections, the Intersection over Union (IoU) with the ground truth exceeded 50%.
- **Localization Precision:** Crucially for ROI calculations, the stricter **mAP@50-95** metric reached **0.752**. This high score confirms that the model does not just detect the *presence* of a logo, but tightly bounds the logo area, ensuring accurate pixel-area calculations for sponsorship valuation.
- **Precision & Recall:** At the best model checkpoint, we achieved a **Precision of 0.936** and a **Recall of 0.880**, minimizing both false positives (hallucinated logos) and false negatives (missed exposures).

Loss Convergence As illustrated in 4, the training losses (Box, Class, and DFL) exhibited a steep initial descent during the first 20 epochs, stabilizing into a steady decay until epoch 100.

- **Box Loss:** Decreased from ~ 1.94 to **0.58**, indicating rapid learning of logo geometry.
- **Classification Loss:** Dropped from ~ 4.05 to **0.30**, showing the model successfully distinguished between the 86 distinct sponsor classes.

Confidence Thresholding We analyzed the F1-Confidence curve 5 to determine the optimal deployment threshold. The model achieves a peak **F1 score of 0.90 at a confidence threshold of 0.445**. This empirically derived threshold informed our inference pipeline configuration, allowing us to filter low-confidence background noise while retaining the vast majority of legitimate brand exposures.

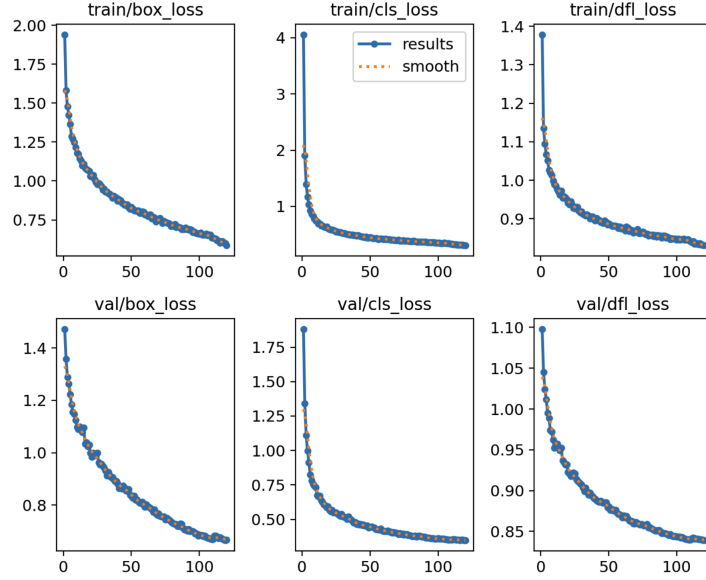


Figure 4: Training and Validation loss curves over 120 epochs. The convergence of validation loss (orange) with training loss (blue) indicates a well-generalized model with minimal overfitting.

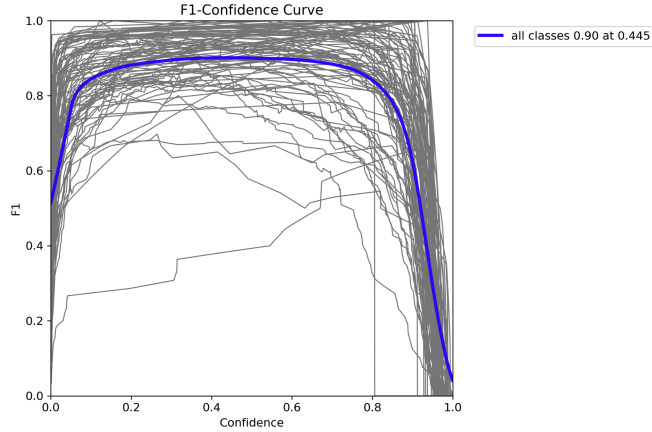


Figure 5: F1-Confidence Curve. The model peaks at an F1 score of 0.90 with a confidence threshold of 0.445, which serves as our baseline for filtering predictions in the production inference pipeline.

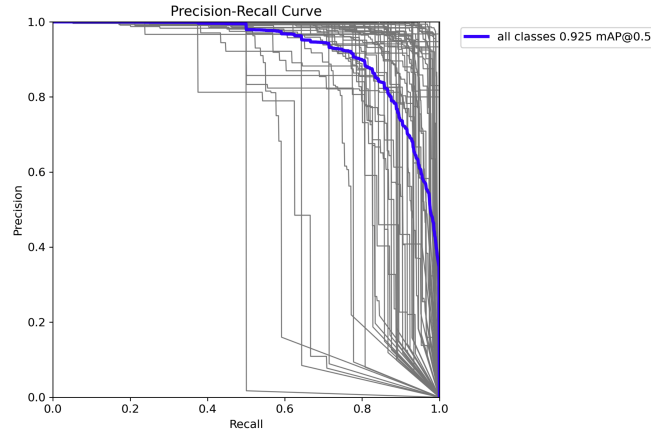


Figure 6: Precision-Recall Curve. The area under the curve (AUC) of 0.925 confirms the model’s robustness in maintaining high precision even as recall sensitivity increases.

5.0.2 Service Scaling

We ran experiments on the video processing speed of the endpoint on different configurations and on a T4 and A10 GPU, each configuration was run 5 times and the averages are reported. We plotted the frames per second of direct model inference versus other processing for each GPU in Figures 7 and 8. Each GPU was run with the maximum batch size that would fit on VRAM (for T4 it was 40, A10 it was 60). All experiments were run on a video with a 10 minute video of 25 fps but stopped inference once the specified frames were reached. However, the T4 results seem to have a slight increase in per frame efficiency in the 14,000 frame runs whereas A10 results have a slight decrease in efficiency. While it is possible for decreases in efficiency to be related to memory overhead, it is strange that the smaller GPU slightly improves with more frames. A more comprehensive study on the GPU utilization and a higher sample experiment should be run to glean more.

Linear frame processing efficiency: Given that each GPU’s experiment was run on a constant batch size, we expected that the model inference runtime would linearly scale with respect to the number of frames (i.e. a constant 0 slope in each of the graphs). It seems that for both GPUs the per frame efficiency becomes constant beyond around 1000 frames. For the inference portion of the processing time when moving from 100 frames to 1000 frames, the latency per frame decreases by $\sim$. This initial lack of processing efficiency can be accounted for by the constant overhead of startup time to transfer and load the model weights into the GPU, or download the video. These findings demonstrate that our inference pipeline remains resilient across a wide range of video lengths without incurring disproportionate overheads. The differences for the non-inference processing follows a similar pattern, where it has an initial drop and then remains constant across the number of frames as expected.

Compute improvements from T4 to A10: Comparing the GPUs, we see that the A10 has an overall 30% decrease in inference overheads as compared to the T4. This is even with the same batch size, the improved FLOPs of A10 and memory bandwidth alone. In addition, although not tested, we can expect an additional 33% decrease in latency since the maximum batch size of the A10 is 60 and not 40 meaning the A10 has fewer batches to compute. In total this would amount to a 53% latency decrease!

GPU underutilization, Non-Inference Latencies are too high: Currently, it takes approximately 35 minutes to run inference on a 25 frames per second, 10 minute video on both GPUs (half of which is spent on inference). Of this, over half of

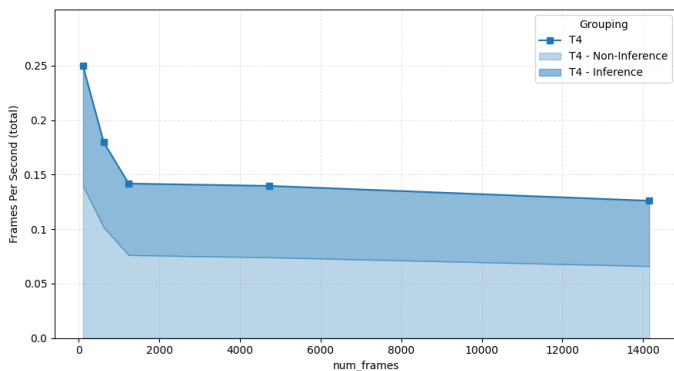


Figure 7: T4 (16GB of memory), max batch size = 40

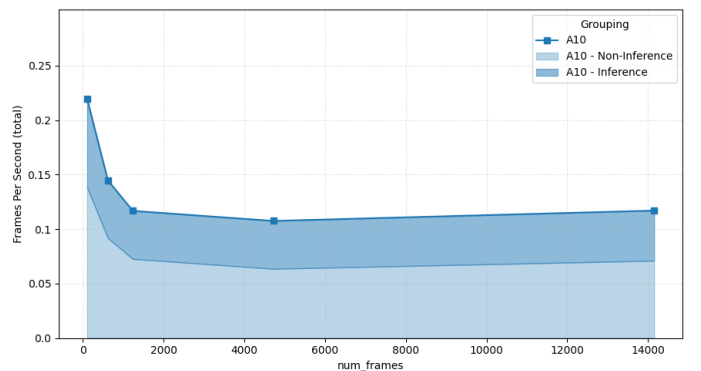


Figure 8: A10 (24GB of memory), max batch size = 40

the processing is from the non-GPU processing latencies. This is a significant area of opportunity to cut on inference costs, particularly when the current inference pipeline is not pipelined, meaning that the GPU is underutilized.

Is it worth scaling? The results show a significant memory bottleneck which limits the benefit of a larger GPU. In terms of Modal’s compute pricing, the A10 costs 83% more than the T4. Since we are only seeing 53% increase in speed, this scale is not worth it. We need to continue with significant improvements to the pipeline and parallelism to better utilize the higher capacity GPUs.

Impact of Optimizations

Initial benchmarks (Figures 9 and 10 in the Appendix) indicated diminishing returns when scaling to larger GPUs (A10) due to the previously mentioned disk I/O bottleneck. Following the implementation of the in-memory TensorRT pipeline, we observed that the “non-inference processing” time dropped to near zero. The system is now limited only by video decoding speed and GPU inference throughput, significantly improving the cost-performance ratio of the T4 instances.

6 Conclusion

6.1 Limitations

While functional for production use, there are several important limitations that constrain accuracy, scalability, and generalizability.

6.1.1 Tailoring Model to Human Perception

Our primary business use-case tries to use these brand detections as a proxy to exposure to the f1 viewing audience. However, our model’s ground truth is the objective ground truth of a presence of a label and not aligned or tuned to a human perceptual ground truth. Future improvements might either try to gather a curated human perceptual dataset or use joint other metrics to proxy recognizability. This requires an in-depth human perceptual study.

6.1.2 Accuracy constraints

Long-Tail Class Performance: Naturally model performance degrades significantly for rarer sponsor classes. Smaller, regional sponsors would simply have less training data.

Motion Blur Sensitivity: Despite the augmentation, the model still struggles with real-world high-speed footage where logos experience severe directional blur. High-speed cornering sequences receive the most attention and these are the moments with the most blur. This is systematic measurement error that underestimates exposure during action-heavy race segments.

6.1.3 Closed-Set Limitation

New Sponsor Adaptation: The current system is completely incapable of adjusting to new sponsors. This creates an operational friction during off-season when lineups change. This supervised approach to labelled sponsor data prevents generalizability to new potential clients of the service.

Seasonal Maintenance: F1 teams typically refresh sponsor packages annually, and turn-over is expected. Maintaining model accuracy requires continuous dataset updates and retraining, requiring annotation and retraining, meaning a lot of compute costs.

6.1.4 Infrastructure and Scalability

Processing Latency: Current throughput averages 0.23 seconds per frame (T4 GPUs, batch size 80), so approximately 55 minutes for a 10-minute video at 6 FPS. While this is acceptable for post-race analysis, this latency prohibits real-time applications. GPU memory constraints prevent effective use of larger batches or more powerful GPUs without significant architecture changes.

Cost Scalability: At current speeds, analyzing a full race weekend for a single sponsor is approximately \$8 – 12 in GPU compute. Scaling to dynamic sets of brands (requiring retraining) across all races could easily exceed thousands of dollars, excluding storage and network transfer fees. This pricing structure limits market accessibility to well-funded sponsors and enterprise analytic firms.

Memory Bottleneck: The sharp performance degradation (slight speedup with a substantial increase in cost) indicates the severe memory bandwidth bottlenecks. As show in Figures 7 and 8, inferences FPS degrades for longer videos, suggesting a memory-related slowdown from buffering or model weight transfers. This architectural limitation prevents efficient use of the high-memory GPUs.

Disaster Recovery Worst-Case: While there is cross-region S3 replication, the inference service itself lacks the geographic redundancy. AWS us-east-1 outage would result in complete service unavailability, as Modal functions and GPU instances are deployed in a single region. The backup S3 bucket provides data recovery but not complete redundancy.

6.2 Future Steps

There are three tiers of future development priorities that can be done: Near-term (optimizations), mid-term (enhancements), long-term (research directions).

6.2.1 Near-term Optimization

1. **Adaptive Batch Sizing:** Replace the fixed batch size with dynamic adjustment based on available memory, frame resolution/complexity, and current memory pressure.
2. **Multi-Region Deployment:** Deploy multiple inference endpoints to provide redundancy, reduced latency for EU user's (F1 primary market), and load balance across regions.
3. **Hardware Escalation:** We are currently benchmarking the optimized pipeline on NVIDIA A10G instances. Preliminary analysis suggests that the increased memory bandwidth of the A10G (600 GB/s vs T4's 320 GB/s) combined with the new streaming architecture will resolve the throughput bottlenecks observed in early tests, validating the cost-benefit ratio for enterprise-tier processing.

6.2.2 Medium-Term Enhancements

1. **Weather-specific Augmentation:** Synthetic weather effects, fog, and other under-represented conditions. Collaborate with F1 media to obtain older footage of these rare races for training data.
2. **Active Learning Pipeline:** Deploy the model in production with confidence-based sampling, automatically flagging low-confidence detections for human review and dataset expansion.
3. **Multi-sport Expansion:** Expand to other sports. Transfer the learned weights and do sport-specific fine-tuning to provide a more expansive service.

6.2.3 Long-term R&D

1. **Live Frame Ingestion:** Accept video live streams directly from broadcasts.
2. **Sub-second Latency:** Achieve <500ms detection-to-display for live dashboards.
3. **Generalizability:** Allow companies to upload sample logos which are immediately augmented and transposed on existing data for training. This would allow sponsors who never worked with F1 to have immediate results using the service.
4. **Complete Restructuring for Generalization:** A potential shift could be creating a model focused on brand logo identification, not specific to F1. Through augmented logo datasets and transposition, we would then have a model capable of computing ROI for any sport and any potential logo.
5. **Semantic Scene Understanding:** Integrate scene classification for more metric data. Detect race events co-occurring with logo visibility, identify camera types to analyze exposure patterns, or correlate logo exposure with on-screen graphics. This would allow queries such as "How much exposure did we receive during top-3 position battles."
6. **Competitive Benchmarking:** A client-facing analytics platform with historical trend analysis, competitive anonymous comparisons, more detailed ROI estimation, and predictive analytics for select semantically-analyzed scenes.

7 Reflection

7.1 Yusuf

As a huge F1 fan, this project was personal for me. My previous computer vision experience was strictly with static images, so moving to video processing was a massive jump in complexity. I quickly learned that training the model is actually the easy part.

The real battle was the infrastructure. I spent a lot of time fighting NVIDIA TensorRT, dealing with cryptic CUDA crashes and GPU architecture mismatches. The most humbling moment was when upgrading to A10G GPUs didn't speed anything up because our code was disk-bound. It taught me a hard lesson that you cannot just throw expensive hardware at unoptimized code. I learned that ML engineering is less about having the smartest model and more about being a really efficient plumber for your data.

Besides the project, I really appreciated the depth and breadth of the lectures. It covered a lot and gave us some really cool insights into what it takes to be a successful ML engineer from planning to architecting to developing an end-to-end solution. Am very grateful for the opportunity to have taken this course!

7.2 Mikhail

Unfortunately I was not familiar with any topics related to the project going in, but coming out I learned the basics of AWS, cloud computing with Modal, hosting vision/LLM models, using LLMs for tests, and optimizing training. I come from a more psychology/cognitive science background in research with limited experience in using services used in software engineering. Working on this project and seeing the codebase develop, contributing where I can, really threw me into real-world machine learning engineering. It equipped with me with the relevant expertise to create a foundation for future learning related to software/ML engineering if I ever decide to pursue that.

Coursework for infrastructure was the most eyeopening one for me. I have always been limited to using the resources available to me on my computer in terms of compute and never threading or redirecting code to the cloud. Accessing AWS S3 buckets, selecting which GPU to use on modal, using docker for environments, and actually running inference without fearing my laptop would explode is the key experience I will be taking with me after this course. Even in research such things are necessary, especially cloud computing.

7.3 Rudraksh

Working on this project was really fun for me because I am a big F1 fan. Thus, being able to work at the intersection of my passion and my profession (Software Engineering / ML Engineering) was fun. On a more technical side, I've worked on end-to-end ML-based projects previously, but this was the first time I had to consider things around usage patterns and heterogeneity w.r.t to result consumption, and I can see those learnings making their way into my future work. On a similar note, this was the first time I had worked with load testing. I knew of the field previously, but getting hands-on experience with Locust and discovering limitations under load was a memorable experience. As noted by my teammates, much of the content of the lectures was more based around LLMs, but other aspects around model serving, Docker, K8s, as well as the guest lectures such as the one by Xavier Portilla Edo, shed a lot of light into what goes on behind the scenes when deploying these ML-based solutions for enterprise, something I can see myself doing in the future with my entrepreneurial ventures.

7.4 Codi

The project was a helpful way to scaffold my learning of the design considerations and constraints of a real, practical ML solution. We were challenged to have a balanced emphasis on model prediction quality and latency. We were also challenged to work within the strict timeline of the course and still produce a "final product", which forced us to learn to be our own product and project managers in this hypothetical startup environment. While much of the lecture content emphasized the LLM industry perspective and our solution followed a more traditional deep learning approach, the lectures helped structure the different steps to the project. Particularly, the infrastructure considerations, product managing crash course, and feature store content are useful regardless of modelling architecture. For example, in choosing our primary service platform to be Modal (serverless), we followed the in-class exercise of matching our solution to our expected randomness and latency requirements.

A Appendix

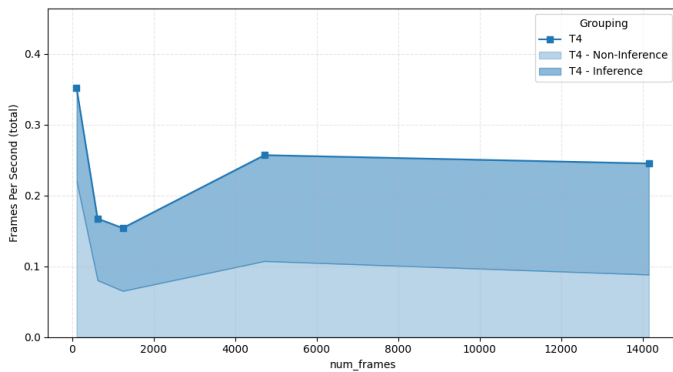


Figure 9: T4 (16GB of memory), max batch size = 80

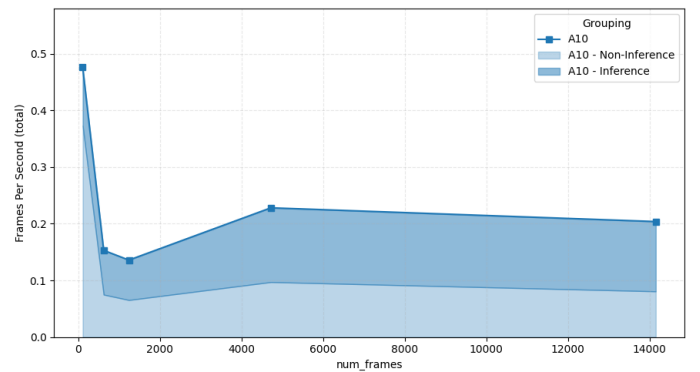


Figure 10: A10 (24GB of memory), max batch size = 400

References

- [1] Clinkscales, J. "F1 Team Sponsor Deals Average 8X More Than NFL, Report Says", *Sportico*, 2025. <https://www.sportico.com/business/sponsorship/2025/f1-team-sponsor-deals-nfl-report-1234854316/>
- [2] Li, Y., Wang, Y., et al. "Open World Object Detection: A Survey", *IEEE Transactions on Circuits and Systems for Video Technology*, 2025. <https://arxiv.org/abs/2410.11301>
- [3] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A., "You Only Look Once: Unified, Real-Time Object Detection", 2016. <https://arxiv.org/abs/1506.02640>
- [4] Cheng, T., Song, L., et al., "YOLO-World: Real-Time Open-Vocabulary Object Detection", 2024. <https://arxiv.org/abs/2401.17270>
- [5] Liu, S., Zeng, Z., et al., "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection", 2024. <https://arxiv.org/abs/2303.05499>
- [6] Carion, N., Massa, F., et al., "End-to-End Object Detection with Transformers", 2020. <https://arxiv.org/abs/2005.12872>
- [7] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," , 2021. *Proceedings of the IEEE/CVF international conference on computer vision*, <https://arxiv.org/abs/2103.14030>
- [8] Kirillov, A., Mintun, E., Ravi, N., et al., "Segment Anything", 2023. <https://arxiv.org/abs/2304.02643>
- [9] Ravi, N., Gabeur, V., et al., "SAM 2: Segment Anything in Images and Videos", 2024. <https://arxiv.org/abs/2408.00714>
- [10] Ren, T., Liu, S., Zeng, A., et al. "Grounded SAM: Assembling Open-World Models for Diverse Visual Tasks", 2024. <https://arxiv.org/abs/2401.14159>
- [11] Lianghui, Z., Ouyang, B., et al. "LENS: Learning to Segment Anything with Unified Reinforced Reasoning", 2025. <https://arxiv.org/abs/2508.14153>
- [12] Liu, Y., Peng, B., et al. "Seg-Zero: Reasoning-Chain Guided Segmentation via Cognitive Reinforcement", 2025. <https://arxiv.org/abs/2503.06520>
- [13] Redmon, J., and Farhadi, A., "YOLO9000: Better, Faster, Stronger", 2016. <https://arxiv.org/abs/1612.08242>
- [14] V. Sharma, M. Gupta, A. Kumar and D. Mishra, "Video Processing Using Deep Learning Techniques: A Systematic Literature Review," *IEEE Access*, 2021. <https://ieeexplore.ieee.org/document/9563948>
- [15] Pattichis, M., Jatla, V., and Ullao Cerna, A., "A Review of Machine Learning Methods Applied to Video Analysis Systems", 2023. <https://arxiv.org/abs/2312.05352>
- [16] AegisVision, "AegisVision Systems for Faster Operations, Accurate Inventory Safe Warehousing", AegisVision Website, <https://www.aegisvision.ai/>
- [17] DragonflyAI Co., "Why marketers choose Dragonfly AI", DragonflyAI Website <https://dragonflyai.co/your-role-marketing>
- [18] Abirami, V., "The AI pit stop at Formula One", 2024. Ultralytics Blog. <https://www.ultralytics.com/blog/the-ai-pit-stop-at-formula-one#f1-teams-and-ai-innovators-a-match-made-on-the-track>
- [19] Memories.AI, "Technical Challenges of AI in Video Analysis", Memories.AI Blog, 2025. https://memories.ai/blogs/Technical_Challenges_of_AI_in_Video_Analysis
- [20] Mukherjee, M., and Hellendoorn, V., "Skill over Scale: The Case for Medium, Domain-Specific Models for SE", 2025. <https://arxiv.org/abs/2306.03268>
- [21] Martineau, K., "Serving customized AI models at scale with LoRA", IBM Blog, 2024. <https://research.ibm.com/blog/LoRAs-explained>
- [22] Hu, E., Shen, Y., Wallis, P., et al., "LoRA: Low-Rank Adaptation of Large Language Models", 2021. <https://arxiv.org/abs/2106.09685>

- [23] Sansverio, O., Tunstall, L., Schmid, P., et al., "Mixture of Experts Explained", 2023. <https://huggingface.co/blog/moe#what-is-a-mixture-of-experts-moe>
- [24] IBM, "Driving the Future of Manufacturing With Edge AI: Accelerating the Revolution", IBM, 2024. <https://www.intel.com/content/dam/www/central-libraries/us/en/documents/2024-06/manufacturing-edge-ai-ebook.pdf>
- [25] Relo Metrics, "Sponsorship Measurement Platform", <https://relometrics.com/sponsorship-measurement-platform>. Computer vision platform for sports sponsorship analytics.
- [26] Blinkfire Analytics, "BrandSpotter: AI-Driven Logo Detection", <https://www.blinkfire.com/landing/mediaanalytics>. Cross-channel brand visibility measurement platform.
- [27] Sponsorlytix, "AI-Powered Sponsor Detection and ROI Measurement", <https://sponsorlytix.ai/>. Real-time sponsor detection system for video and esports.
- [28] Locust Contributors, "Locust: An Open Source Load Testing Tool", 2024. <https://docs.locust.io/>. Python-based load testing framework.